

Paper - 01

Guía práctica para usar metodología Skills en el desarrollo aplicaciones y juegos

Autor: Ing. Ronny Díaz López

Quito, Ecuador

Martes, 23 de Junio de 2026

Esta guía transforma la metodología de "Skills" (sin necesidad de Multiagentes) de Anthropic usada en Claude.ai para ser aplicada utilizando el framework **Free OpenCode** (u otros entornos de código abierto), utilizando modelos de IA gratuitos (como Minimax, Big Pickel, Ollama, DeepSeek V4 Flash Free, MiMo v2.5, North Mini Code Free, Nemotron 3 Ultra, y más) para desarrollar y expandir por ejemplo este caso de uso: "Juego **Neon Dash**".

Opencode es un framework gratuito (con planes de pagos también) para el desarrollo de proyectos, opensouce, con opciones de desarrollo usando modelo tanto gratuitos como de pago. Estos modelos ya han sido "curados" probados por Opencode así que es un plus en el desarrollo de aplicaciones y demás proyectos. O mejor aún cómo ellos mismos lo dicen en su web: "Un agente de código abierto que te ayuda a escribir código en tu terminal, IDE o escritorio." (<https://opencode.ai/>)

Mapa de Implementación en OpenCode (así lo uso yo diariamente)

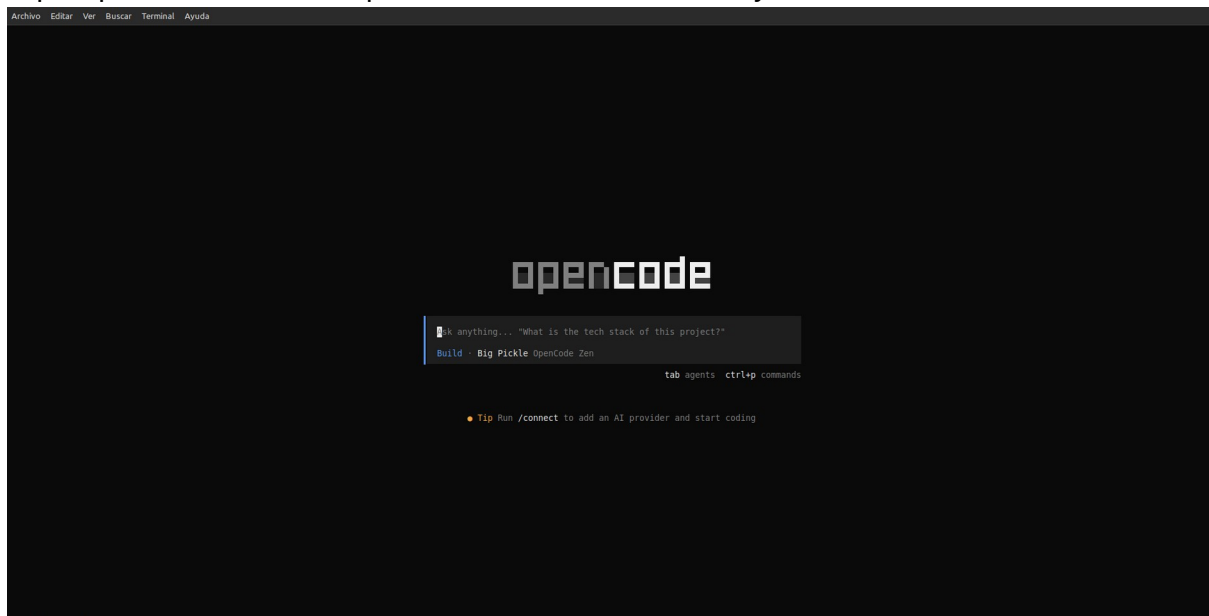
```
graph TD
  A[Analizar Código Base Neon Dash] --> B[Definir 'Agente Dev' en OpenCode]
  B --> C[Crear System Prompt / Skill .md]
  C --> D[Pruebas de Refactorización]
  D --> E{¿Funciona el Juego?}
  E -->|No| F[Iterar Prompt con Errores]
  F --> D
  E -->|Sí| G[Guardar en Git para el Equipo]
```

Paso 1: Definición del "Agente de Desarrollo"

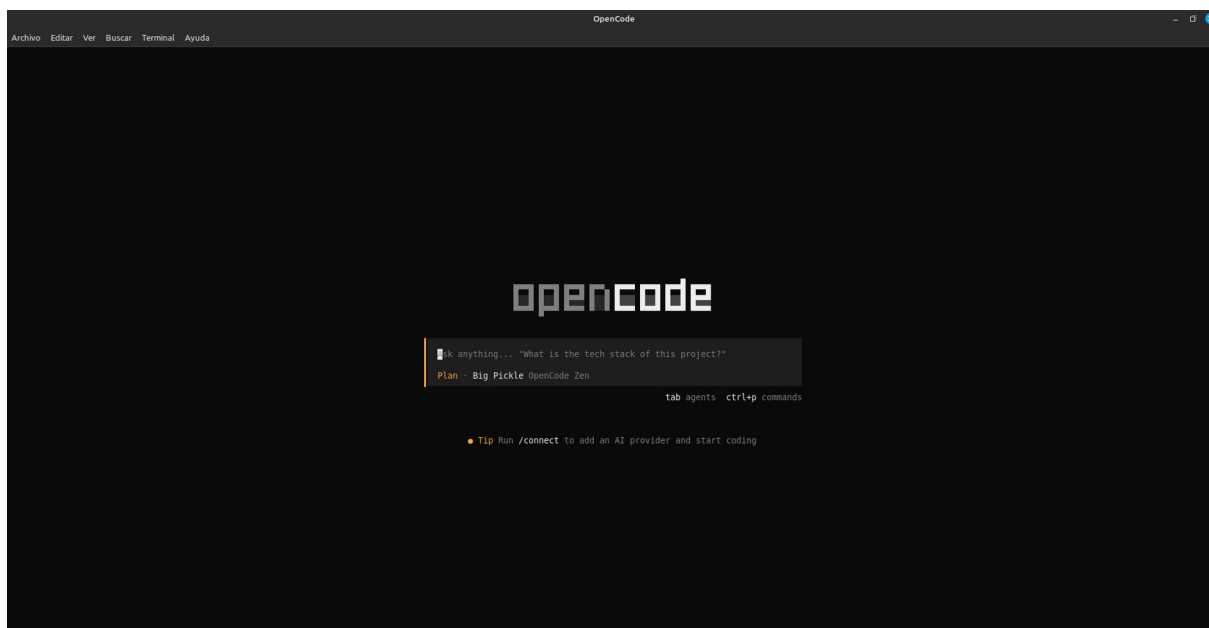
En lugar de un prompt efímero o de estar creando a diestra y siniestra,, gastando balas (tokens) por gusto, en AgenticDevs o programación Multiagentes sin necesidad alguna excepto para proyectos grandes que ameriten la pena usar este tipo de programación avanzada con HITL (Human-In-The-Loop – Humanos supervisando el bucle o lazo) porque te recomiendo que nunca dejes a la IA hacer y tomar decisiones por sí misma sin tu revisar, aprobar y supervisar lo que va a hacer antes de hacerlo, entonces crearemos una **Skill persistente** que actúe como el arquitecto jefe de "Neon Dash".

Para abrir Opencode (no tienes necesidad de usar por el momento ningún editor de código como VS Code, o en mi caso uso VS Codium, que no tiene telemetría ni vende tus datos a terceros como si hace VS Code). TE ubicas dentro de tu carpeta raíz del proyecto (Ejemplo del juego que estamos desarrollando: neon-dash) y si estas en Linux Mint como yo aprietas Ctrl + Alt + T y se te abrirá la terminal de linux ya en la ruta de tu proyecto. Y entonces escribes o ejecutas: opencode

Aquí OpenCode te muestra por defecto dos modos: Build y Plan.



Y si aprietas la tecla Tab cambias de modo:



En mi caso primero recomiendo comenzar por el modo “Plan” para que OpenCode te ayude a planificar toda la arquitectura y detalles de tu proyecto, y vas revisando e iterando y cuando ya tengas pulido o listo todo el plan de desarrollo de tu proyecto, que te satisfaga, entonces le dices: “Procedamos con la implementación” pasando al modo Build (aprietas la tecla Tab) y con esto ya tienes abierto el entorno de desarrollo de OpenCode!!

En OpenCode, modo Plan, configurarás un "System Prompt" que incluya las reglas de oro de tu juego.

- **Tu frase mágica de inicio:** Al configurar el agente, dile: *"Eres el Lead Developer del juego Neon Dash. Basándote en el código disponible en internet o (Aquí si tienes algo de código como es mi caso lo subes) actual, ayúdame a (X) funcionalidad. ¿Qué más debería aclararte antes de empezar?"*

Con este tipo de pedido a OpenCode le pones un primer freno a la IA, porque no la dejas ir a meter código de una vez sino primero revisarlo tú para aprobar o rechazar o modificar algo.

Paso 2: Estructura de la Skill (Archivo .md)

Siguiendo la metodología de "Front Matter" y "Body" [Ref. 1], creas dentro de tu carpeta raíz del proyecto, un archivo llamado "neon-dash-architect.md". Este archivo es el archivo base a partir del cual desarrollarás más adelante con ayuda de OpenCode todo el proyecto incluyendo el SKILL.md, AGENTS.md, etc.

NOTA: Te recomiendo que donde dice debajo "name: neon-dash-architect" dentro de tu archivo .md le des ese mismo nombre a tu carpeta del proyecto raíz para mantener coherencia en el desarrollo de tu aplicación o proyecto.

Ejemplo de Estructura:

```
---
name: neon-dash-architect
model: Big Pickel (OpenCode)
description: Experto en la lógica, frontend y estética de Neon Dash.
---
# Contexto Técnico del Juego
- Gravedad: 0.55 | Salto: -9.
- Estética: Neon/Arcade con colores `#00f0ff` (cyan) y `#ff0066` (magenta).
- Estado: Maneja los estados `start`, `playing` y `dead`.

# Reglas de Negocio
1. Toda nueva función debe respetar la colisión `rectsOverlap`.
2. Los efectos visuales deben usar el sistema de partículas `emitParticles`.
```

Paso 3: Integración del Código Base (RAG Manual)

Para que la Skill sea "específica" (que es el ingrediente clave del éxito), debes alimentar al modelo con los fragmentos críticos del archivo .docx: (Este archivo que yo le llamo "codigo_neon_dash.docx", es dónde en mi caso que ya tenía archivos básicos con código HTML, CSS y JS los pegué allí dentro y se los adjunto o coloco dentro de la carpeta raíz del proyecto. Si en tu caso no tienes esto como base no importa recuerda que la IA es experta en generar código a la medida, se lo pides o indicas que te los genere dentro de tu archivo "neon-dash-architect.md" y listo).

- **Lógica de Movimiento:** Enséñale que el jugador usa HOVER_THRUST para subir y la gravedad para caer.

- **Obstáculos:** Muéstrale que los obstáculos aparecen en intervalos que se reducen según el puntaje (SPAWN_INTERVAL).
- **Audio:** Recuérdale que el juego usa AudioContext para generar sonidos sintéticos como playJump y playDeath.

Paso 4: Pruebas e Iteración en OpenCode

No asumas que la Skill es perfecta. Pruébala pidiendo una nueva funcionalidad, por ejemplo: *"Crea un nuevo tipo de obstáculo que se mueva en zigzag"*.

- **Si el código falla:** Copia el error de la consola y dile a la Skill: *"Ajusta tu comportamiento, la función drawObstacles está causando un parpadeo"*.
- **Filtro de Calidad:** Asegúrate de que la Skill no invente variables; siempre debe referenciar a canvas, ctx o player del código base.

Paso 5: Compartir y Colaborar (Modo Git)

Luego debes subir tu desarrollo del proyecto a un Repositorio como Github, si deseas compartirlo con tus amigos u otras personas, lo pones público y si no pues privado. Es tu elección. Usa esta metodología mediante **Git/GitHub**.

Una vez que ya has refinado y revisado una y otra vez el plan propuesto por OpenCode, pasas al modo Build y procedes a:

1. Crear otra carpeta dentro de tu carpeta raíz del proyecto llamada ".opencode" y dentro de esta creas otra carpeta llamada "skills".
2. Colocas allí dentro el archivo "neon-dash-architect.md".
3. Eso mismo repites o subes con Git a tu repo en Github. Si lo quieres compartir con tus amigos solo tienen que clonar el repositorio.

Al abrir OpenCode en esa carpeta, el framework detectará automáticamente la Skill y podrán invocarla escribiendo /neon-dash en el chat.

Ejemplo de Aplicación: "El Cazador de Bugs Neon"

Puedes usar esta Skill para auditar tu código. Si sientes que el juego es muy difícil, invoca la Skill y pregunta: *"¿Cómo puedo ajustar la curva de dificultad (diff) para que el gapSize no se reduzca tan rápido?"*

La Skill sabrá exactamente dónde tocar el código en script.js sin que tú tengas que buscar línea por línea.

Aquí les dejo el enlace a mi github para que puedan descargar y usar todo este contenido a su criterio y responsabilidad:

Repositorio "neon-dash-architect", enlace github: <https://github.com/ecoruralia/neon-dash-architect.git>

Te deseo éxitos en lo que sea que hagas!!

Un saludo Ingenieril Digitalizado Rural

Escríbeme contándome Qué te pareció? O al menos agradeciendo si te fue útil.

Email: ronnydldev@gmail.com

Bibliografía:

[Ref. 1] La metodología de "Front Matter" (materia frontal) y "Body" (cuerpo) es un estándar universal para la creación y gestión de contenidos en el desarrollo web moderno, blogs y lenguajes de marcado (como Markdown o XML JATS). Su objetivo es separar los metadatos de la información principal para automatizar el diseño y mejorar la organización.

Componentes Principales:

1. Front Matter (Metadatos): Es el bloque de configuración inicial del documento, generalmente ubicado en la parte superior y delimitado por tres guiones (---). Contiene información estructurada sobre el archivo, pero no el texto final que leerá el usuario.

- Formato: Normalmente escrito en YAML, JSON o TOML.
- Uso: Define variables clave que los motores de plantillas leen automáticamente.
- Ejemplos de campos: Título, autor, fecha de publicación, etiquetas (tags), descripción para SEO, imagen destacada o permisos.

2. Body (Cuerpo): Es el contenido central y narrativo del documento. Contiene la información real que se mostrará al usuario final.

- Formato: Escrito en lenguajes de texto enriquecido o marcado, principalmente Markdown para sitios web, o XML JATS para artículos científicos.
- Uso: Estructura la información mediante párrafos, listas, citas, enlaces y encabezados.

¿Cómo interactúan en la práctica?

En sistemas y generadores de sitios estáticos (como Hugo, Jekyll o Eleventy), un procesador lee el archivo y divide ambas partes de manera programática:

1. Lectura: El sistema lee el Front Matter para saber cómo debe comportarse la página.
2. Generación de la cabecera HTML: Utiliza los datos del Front Matter para rellenar las etiquetas <title>, <meta description>, y las propiedades Open Graph de redes sociales.
3. Conversión del Body: El sistema compila el Body desde Markdown a HTML limpio y lo inyecta en la estructura principal de la página (ej. en la etiqueta <body>).

Ejemplo Práctico: Un artículo de blog típico tiene esta estructura dentro de un archivo de texto .md:

```
---
title: "Guía de Metodología Front Matter"
date: 2026-06-23
author: "Equipo Editorial"
tags: ["Desarrollo Web", "Markdown"]
description: "Aprende a separar metadatos y contenido con esta guía."
---
```

Introducción

El contenido principal del artículo comienza aquí. El cuerpo puede tener ****negritas****, listas y enlaces.

Siguiendo pasos

Ahora que conoces la estructura, puedes implementarla en tus proyectos...